

Day-10 Hands-on (PostMan)

Starting the server :

```
C:\Users\engua\Desktop\coforge-training\day-10\hands-on>json-server db.json --port 3000
JSON Server started on PORT :3000
Press CTRL-C to stop
Watching db.json...

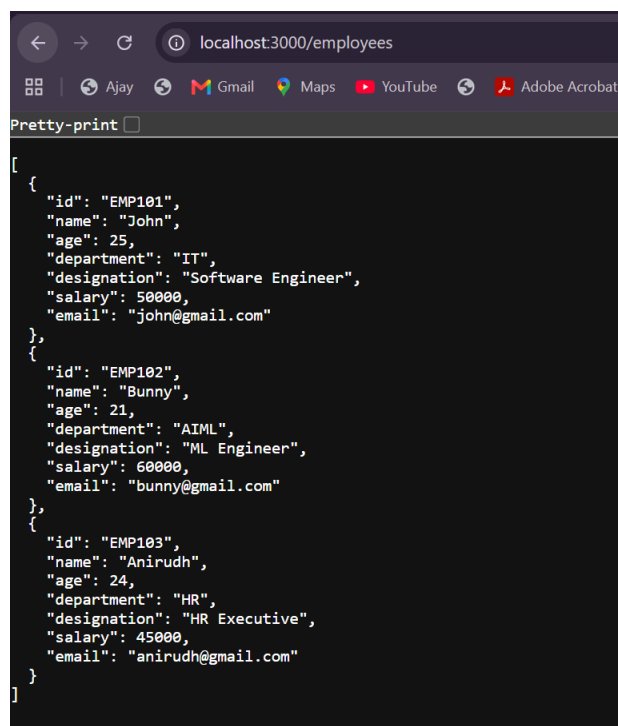
(^ ^ ~ ^ ^)

Index:
http://localhost:3000/

Static files:
Serving ./public directory if it exists

Endpoints:
http://localhost:3000/employees
```

Data:



A screenshot of a web browser window displaying the JSON data for the `localhost:3000/employees` endpoint. The browser's address bar shows the URL. Below the address bar, there are several icons for various services like Ajay, Gmail, Maps, YouTube, and Adobe Acrobat. The main content area shows the JSON data in a dark-themed editor with a 'Pretty-print' button. The data is an array of three employee objects.

```
[
  {
    "id": "EMP101",
    "name": "John",
    "age": 25,
    "department": "IT",
    "designation": "Software Engineer",
    "salary": 50000,
    "email": "john@gmail.com"
  },
  {
    "id": "EMP102",
    "name": "Bunny",
    "age": 21,
    "department": "AIML",
    "designation": "ML Engineer",
    "salary": 60000,
    "email": "bunny@gmail.com"
  },
  {
    "id": "EMP103",
    "name": "Anirudh",
    "age": 24,
    "department": "HR",
    "designation": "HR Executive",
    "salary": 45000,
    "email": "anirudh@gmail.com"
  }
]
```

GET All Employees:

My Collection > Get data

GET http://localhost:3000/employees

200 OK • 8 ms • 861 B • Save Response

```
{
  "id": "EMP101",
  "name": "John",
  "age": 25,
  "department": "IT",
  "designation": "Software Engineer",
  "salary": 50000,
  "email": "john@gmail.com"
},
{
  "id": "EMP102",
  "name": "Bunny",
  "age": 21,
  "department": "AIML",
  "designation": "ML Engineer",
  "salary": 60000,
  "email": "bunny@gmail.com"
},
{
  "id": "EMP103",
  "name": "Anirudh",
  "age": 24,
```

Designing a new onb...
/onboard-user Help me design a new API.
get:
summary: List items
responses:
200:
description:
Successful
response
If you want, I can help you do one of these next:
• draft a first OpenAPI spec for your API
• design the resources and endpoints with you
• turn your idea into a collection structure in Postman
Tell me your API idea in one sentence, for example: I want an API for a bookstore with books, customers, and orders.
Describe what you need. Press @ for context, / for Skills.
Auto

GET Employee by ID:

My Collection > Get data

GET http://localhost:3000/employees/EMP101

200 OK • 9 ms • 482 B • Save Response

```
{
  "id": "EMP101",
  "name": "John",
  "age": 25,
  "department": "IT",
  "designation": "Software Engineer",
  "salary": 50000,
  "email": "john@gmail.com"
}
```

POST (Add New Employee):

The screenshot shows the Postman interface for a POST request. The URL is `http://localhost:3000/employees`. The request body is a JSON object with the following fields: `id` ("EMP103"), `name` ("Rahul"), and `age` (26). The response status is **201 Created**, with a response time of 11 ms and a size of 492 B. The response body is a JSON object with the following fields: `id` ("yHQJFMkKFni"), `name` ("Rahul"), `age` (26), `department` ("Finance"), `designation` ("Accountant"), `salary` (55000), and `email` ("rahul@gmail.com").

```
{
  "id": "EMP103",
  "name": "Rahul",
  "age": 26,
  "department": "Finance",
  "designation": "Accountant",
  "salary": 55000,
  "email": "rahul@gmail.com"
}
```

Verify Employee Added

The screenshot shows the Postman interface for a GET request. The URL is `http://localhost:3000/employees`. The response status is **200 OK**, with a response time of 9 ms and a size of 1.33 KB. The response body is a JSON array of three employee objects. The first object has `id` "EMP101", `name` "John", `age` 25, `department` "IT", `designation` "Software Engineer", `salary` 50000, and `email` "john@gmail.com". The second object has `id` "EMP102", `name` "Bunny", `age` 21, `department` "AIML", `designation` "ML Engineer", `salary` 60000, and `email` "bunny@gmail.com". The third object has `id` "EMP103", `name` "Anirudh", `age` 24, `department` "HR", `designation` "HR Executive", `salary` 45000, and `email` "anirudh@gmail.com".

```
[
  {
    "id": "EMP101",
    "name": "John",
    "age": 25,
    "department": "IT",
    "designation": "Software Engineer",
    "salary": 50000,
    "email": "john@gmail.com"
  },
  {
    "id": "EMP102",
    "name": "Bunny",
    "age": 21,
    "department": "AIML",
    "designation": "ML Engineer",
    "salary": 60000,
    "email": "bunny@gmail.com"
  },
  {
    "id": "EMP103",
    "name": "Anirudh",
    "age": 24,
    "department": "HR",
    "designation": "HR Executive",
    "salary": 45000,
    "email": "anirudh@gmail.com"
  }
]
```

PUT (Replace Entire Employee)

The screenshot shows a REST client interface with a PUT request configured. The URL is `http://localhost:3000/employees/EMP103`. The request body is a JSON object representing an employee.

```
1 {
2   "id": "EMP103",
3   "name": "Rahul Kumar",
4   "age": 27,
5   "department": "Finance",
6   "designation": "Senior Accountant",
7   "salary": 65000,
8   "email": "rahulkumar@gmail.com"
9 }
```

PATCH (Update Selected Fields):

The screenshot shows a REST client interface with a PATCH request configured. The URL is `http://localhost:3000/employees/EMP103`. The request body is a JSON object containing only the fields to be updated.

```
1 {
2   "salary": 70000,
3   "designation": "Finance Manager"
4 }
```

Below the request body, the response is shown. It is a 200 OK status with a response time of 12 ms and a body size of 498 B. The response body is a JSON object representing the updated employee.

```
1 {
2   "id": "EMP103",
3   "name": "Rahul Kumar",
4   "age": 27,
5   "department": "Finance",
6   "designation": "Finance Manager",
7   "salary": 70000,
8   "email": "rahulkumar@gmail.com"
9 }
```

DELETE Employee:

POST Get data

DEL Get data

+

No environment

My Collection > Get data

Save

Share

DELETE

http://localhost:3000/employees/EMP103

Send

Docs

Params

Authorization

Headers 8

Body

Scripts

Tests

Settings

Cookies

Body

200 OK

6 ms

1.15 KB

Save Response

JSON

Preview

Visualize

```
1  [
2    {
3      "id": "EMP101",
4      "name": "John",
5      "age": 25,
6      "department": "IT",
7      "designation": "Software Engineer",
8      "salary": 50000,
9      "email": "john@gmail.com"
10   },
11   {
12     "id": "EMP102",
13     "name": "Bunny",
14     "age": 21,
15     "department": "AIML",
16     "designation": "ML Engineer",
17     "salary": 60000,
18     "email": "bunny@gmail.com"
19   },
20   {
21     "sid": 102,
22     "sname": "Bunny",
23     "semail": "bunny@gmail.com",
24     "id": "vv8n3iVEhBM"
25   },
26   {
```